

Relazione Tecnica di Progettazione e Sviluppo

Travel Organizer Offline

App mobile per l'organizzazione di viaggi in modalità offline

Anno Accademico 2025/2026

Gruppo 5 — Traccia progettuale n. 1

Crisci Antonio — 0612609486

Oberto Leonardo Pio Pompeo — 0622706053

Mobile Programming

Codice sorgente: <https://github.com/acrisci05/TravelOrganizerOffline>

Indice

1	Descrizione dell'app	2
1.1	Obiettivo dell'applicazione	2
1.2	Tipologia di utenti	2
1.3	Problema che l'app intende risolvere	2
1.4	Principali scenari d'uso	2
2	Requisiti	2
2.1	Requisiti funzionali e modelli dati	2
2.2	Funzionalità implementate	3
2.3	Funzionalità considerate ma non implementate	3
2.4	Feature avanzate scelte	4
2.5	Limitazioni note	4
3	Requisiti non funzionali e vincoli	4
4	Casi d'uso	4
4.1	Elenco dei casi d'uso	4
4.2	Descrizione dettagliata dei casi d'uso	5
5	Progettazione dell'app	6
5.1	Struttura generale	6
5.2	Schermate principali	6
5.3	Flusso di navigazione	6
5.4	Organizzazione dei dati	7
6	Scelte tecnologiche	7
6.1	Framework	7
6.2	Librerie principali	7
6.3	Motivazione delle scelte	8
6.4	Gestione della persistenza	8
7	Implementazione	8
7.1	Organizzazione del codice	8
7.2	Gestione dello stato	8
7.3	Gestione della navigazione	8
7.4	Gestione dei dati persistenti	8
7.5	Parti significative	8
8	Conclusioni	10

1 Descrizione dell'app

1.1 Obiettivo dell'applicazione

Travel Organizer Offline è un'applicazione mobile che aiuta l'utente a pianificare e gestire i propri viaggi mantenendo tutte le informazioni disponibili *anche in assenza di connessione di rete*. L'app consente di organizzare uno o più viaggi, suddividerli in tappe o giornate, associare attività a luoghi e date, gestire checklist e informazioni utili e monitorare le spese previste ed effettive.

L'obiettivo è offrire un unico strumento coerente e usabile che accompagni l'utente in tutte le fasi del viaggio: dalla pianificazione iniziale, alla gestione durante lo svolgimento, fino al consuntivo finale delle spese.

1.2 Tipologia di utenti

L'applicazione è rivolta a viaggiatori occasionali e abituali che desiderano organizzare autonomamente i propri viaggi senza dipendere da servizi online, ad esempio in situazioni di roaming assente o costoso. Non è richiesta alcuna competenza tecnica: l'interfaccia è pensata per essere immediata e guidata.

1.3 Problema che l'app intende risolvere

Le informazioni relative a un viaggio (itinerario, prenotazioni, cose da portare, budget) sono tipicamente sparse tra più applicazioni e servizi, spesso raggiungibili solo con una connessione attiva. Travel Organizer Offline centralizza questi dati in un'unica app che funziona *completamente in locale*, garantendone la disponibilità in ogni momento.

1.4 Principali scenari d'uso

- **Pianificazione di un nuovo viaggio:** l'utente crea un viaggio, ne definisce le tappe, vi associa le attività e prepara la lista valigia.
- **Gestione durante il viaggio:** l'utente consulta la timeline giornaliera, spunta le attività svolte e gli oggetti in valigia, registra le spese.
- **Controllo del budget:** l'utente confronta le spese effettive con il budget previsto e analizza la ripartizione per categoria.
- **Riutilizzo di un itinerario:** l'utente duplica un viaggio già organizzato per pianificarne rapidamente uno analogo.

2 Requisiti

2.1 Requisiti funzionali e modelli dati

Dettaglio modelli dati (DF-1):

- **Trip:** id, title, destination, startDate, endDate, description, status, budget, participants, notes, createdAt.
- **Stage:** id, tripId, title, date, location, description, order, notes.
- **Activity:** id, tripId, stageId, title, description, dateTime, location, category, estimatedCost, status, notes.
- **Checklist:** id, tripId, stageId, title, description; **ChecklistItem:** id, checklistId, title, isCompleted, order.
- **Expense:** id, tripId, stageId, activityId, title, amount, category, date, paymentMethod, status, notes.

Dettaglio funzioni individuali (IF):

Categoria	ID	Descrizione	Priorità
Data Format	DF-1	Strutturazione entità e persistenza locale (Viaggi, Tappe, Attività, Checklist, Spese)	Must
Ind. Function	IF-1	Gestione dei viaggi (CRUD e dettaglio)	Must
Ind. Function	IF-2	Gestione delle tappe/giornate	Must
Ind. Function	IF-3	Gestione delle attività e dell'itinerario	Must
Ind. Function	IF-4	Gestione delle checklist e degli oggetti da portare	Must
Ind. Function	IF-5	Gestione delle spese previste ed effettive	Must
Ind. Function	IF-6	Ricerca, filtri e ordinamento	Should
Ind. Function	IF-7	Riepiloghi e statistiche	Should
Ind. Function	IF-8	Feature avanzate (duplicazione, packing list, timeline)	Should

- **IF-1 Viaggi:** visualizzare l'elenco, consultare il dettaglio, inserire, modificare ed eliminare un viaggio. Lo stato (futuro, in corso, completato, archiviato) è calcolato automaticamente dalle date, con possibilità di archiviazione manuale.
- **IF-2 Tappe:** visualizzare, aggiungere, modificare, rimuovere e riordinare (drag & drop) le tappe; consultare le attività associate a ciascuna tappa.
- **IF-3 Attività:** inserire, modificare, eliminare e cambiare stato (da fare, completata, annullata) di un'attività, associandola al viaggio e, facoltativamente, a una tappa; categoria, data/ora, luogo e costo previsto.
- **IF-4 Checklist:** creare più checklist per viaggio, aggiungere/modificare/eliminare elementi e marcarli come completati; barra di avanzamento.
- **IF-5 Spese:** inserire, modificare ed eliminare spese distinguendo tra previste ed effettive; categoria, metodo di pagamento, tappa/attività associata.
- **IF-6 Ricerca e filtri:** ricerca viaggi per titolo/destinazione; filtro viaggi per stato; ricerca/filtro tappe per nome, località e data; filtro attività per categoria, stato e giorno; filtro spese per stato, categoria e fascia di importo.
- **IF-7 Statistiche:** numero di viaggi per stato, attività totali e completate, spese effettive e previste, avanzamento delle checklist, grafico a torta delle spese per categoria, confronto budget/spesa e classifica delle tappe con più attività.
- **IF-8 Feature avanzate:** descritte nella Sezione 2.4.

2.2 Funzionalità implementate

Sono state implementate tutte le funzionalità richieste dalla traccia: gestione di viaggi, tappe/giornate, attività/itinerario, checklist e informazioni utili, spese, oltre a ricerca/filtri, riepiloghi/statistiche e due feature avanzate. Tutte le entità principali supportano inserimento, visualizzazione, modifica e cancellazione, con persistenza locale.

2.3 Funzionalità considerate ma non implementate

- **Notifiche/promemoria** legate alle date del viaggio: non richieste dalla traccia e non essenziali al dominio offline.
- **Gestione multivaluta e cambio:** come consentito dalla traccia, gli importi sono trattati in un'unica valuta (euro).
- **Suddivisione delle spese tra partecipanti:** i partecipanti sono registrati come informazione testuale, senza calcolo dei saldi individuali.

- **Sincronizzazione cloud / backup remoto:** esclusa per scelta progettuale (app 100% offline).

2.4 Feature avanzate scelte

Sono state realizzate e integrate nell'app le seguenti feature avanzate:

1. **Duplicazione completa del viaggio:** la copia di un viaggio include tappe, attività e checklist, con corretta rimappatura delle associazioni tappa-attività e tappa-checklist.
2. **Packing list generata dal tipo di viaggio:** la lista valigia viene generata automaticamente in base al tipo di viaggio scelto dall'utente (mare, montagna, città, business, avventura, inverno), combinando oggetti di base e oggetti specifici del tipo.
3. **Timeline giornaliera dell'itinerario:** vista cronologica che aggrega tappe e attività ordinate per data (feature aggiuntiva).

2.5 Limitazioni note

- I dati risiedono unicamente sul dispositivo: la disinstallazione dell'app o la cancellazione dei dati applicativi comporta la perdita delle informazioni.
- Le spese non ereditano automaticamente il costo previsto delle attività: si tratta di due grandezze distinte (stima dell'attività vs. spesa registrata).
- La duplicazione non copia le spese: il viaggio duplicato parte con un registro spese vuoto, coerentemente con una nuova pianificazione.

3 Requisiti non funzionali e vincoli

Categoria	ID	Descrizione	Priorità
User Interface	UI-1	Navigazione a più schermate con passaggio dati e feed-back visivo (Material 3)	Must
User Interface	UI-2	Empty state e validazione dei form con messaggi contestuali	Should
User Interface	UI-3	Conferma esplicita delle eliminazioni (AlertDialog)	Must
Constraint	FC-1	Sviluppo in Flutter, cross-platform	Must
Constraint	FC-2	Persistenza locale e funzionamento 100% offline	Must
Constraint	FC-3	Liste performanti tramite <code>ListView.builder</code> (lazy loading)	Should

- **UI-1:** l'app usa una `NavigationBar` inferiore (Viaggi, Statistiche) e una navigazione a schede all'interno del dettaglio del viaggio; il passaggio dei dati tra schermate avviene per parametri costruttore (es. `tripId`).
- **UI-2/UI-3:** i form validano i campi obbligatori con messaggi inline; le eliminazioni richiedono conferma tramite dialog dedicato e i salvataggi mostrano riscontri via `SnackBar`.
- **FC-2:** nessuna chiamata di rete né backend; tutti i dati sono gestiti da un database SQLite locale.

4 Casi d'uso

4.1 Elenco dei casi d'uso

ID	Nome	ID	Nome
UC-1	Inserisci viaggio	UC-6	Registra spesa
UC-2	Consulta dettaglio viaggio	UC-7	Cerca e filtra elementi
UC-3	Inserisci tappa	UC-8	Visualizza statistiche
UC-4	Inserisci attività	UC-9	Duplica viaggio
UC-5	Genera lista valigia	UC-10	Modifica o elimina elemento

4.2 Descrizione dettagliata dei casi d'uso

UC-1: Inserisci viaggio

Attore: Utente.

Precondizioni: app avviata, persistenza inizializzata.

Postcondizioni: un nuovo viaggio è salvato e visibile nell'elenco.

Flusso base:

1. L'utente seleziona "Nuovo viaggio".
2. Il sistema mostra il form di inserimento.
3. L'utente compila titolo, destinazione, date, budget, partecipanti e note.
4. Il sistema valida i campi obbligatori e la coerenza delle date.
5. Il sistema salva il viaggio nel database e aggiorna la lista.

Flusso alternativo (4a) — dati non validi: il sistema evidenzia il campo errato e blocca il salvataggio finché l'input non è corretto.

Flusso alternativo (4b) — date nel passato: il sistema chiede conferma esplicita prima di procedere (permette di registrare un viaggio già svolto).

UC-4: Inserisci attività

Attore: Utente.

Precondizioni: esiste almeno un viaggio.

Postcondizioni: l'attività è associata al viaggio e, facoltativamente, a una tappa.

Flusso base:

1. L'utente apre la scheda "Attività" del viaggio e seleziona l'aggiunta.
2. L'utente inserisce titolo, categoria, data/ora, luogo, costo, tappa e note.
3. Il sistema valida i dati e, se è associata una tappa, verifica che la data dell'attività ricada nel giorno della tappa.
4. Il sistema salva l'attività con stato iniziale "da fare".

Flusso alternativo (3a): data incoerente con la tappa: il sistema segnala l'errore e richiede una nuova selezione di data/ora.

UC-5: Genera lista valigia (feature avanzata)

Attore: Utente.

Precondizioni: è aperto il dettaglio di un viaggio.

Postcondizioni: la lista valigia è popolata con oggetti coerenti al tipo scelto.

Flusso base:

1. L'utente apre la scheda "Valigia" e seleziona "Genera".
2. Il sistema mostra un menu con i tipi di viaggio disponibili.
3. L'utente sceglie il tipo (es. Mare).
4. Il sistema aggiunge alla checklist gli oggetti di base più quelli specifici del tipo, evitando i duplicati già presenti.

UC-9: Duplica viaggio (feature avanzata)

Attore: Utente.

Precondizioni: esiste il viaggio da duplicare.

Postcondizioni: viene creato un nuovo viaggio “(copia)” con tappe, attività e checklist duplicate.

Flusso base:

1. L'utente seleziona “Duplica” su un viaggio.
2. Il sistema crea il nuovo viaggio (stato “futuro”).
3. Il sistema duplica le tappe generando una mappa id originale→id copia.
4. Il sistema duplica attività e checklist ricollegandole alle nuove tappe tramite la mappa.
5. Il sistema conferma l'operazione con una **SnackBar**.

UC-10: Modifica o elimina elemento

Attore: Utente.

Flusso base: l'utente sceglie “Modifica” o “Elimina” su viaggio, tappa, attività, checklist o spesa; nel caso di eliminazione il sistema chiede conferma tramite dialog e, se confermato, aggiorna il database e la vista. La cancellazione di un viaggio elimina in cascata tutti i dati collegati.

5 Progettazione dell'app

5.1 Struttura generale

L'app segue un'architettura a livelli con separazione delle responsabilità: *modelli* di dominio, *repository* per l'accesso ai dati, *provider* per la gestione dello stato e *feature* (schermate) per la presentazione. I widget riutilizzabili sono raccolti nel livello **shared**.

```
lib/  
  core/ tema, colori e utility (date/valuta)  
  data/  
    models/ modelli di dominio  
    database/ DatabaseHelper (SQLite: schema, indici)  
    repositories/ accesso ai dati (una repository per entità)  
    seed/ dati demo iniziali  
  providers/ gestione dello stato (ChangeNotifier)  
  features/ schermate per funzionalità  
  shared/ widget riutilizzabili
```

5.2 Schermate principali

- **Lista viaggi:** elenco ordinato per stato, ricerca, filtri, vista archiviati e azioni rapide (duplica, modifica, elimina).
- **Dettaglio viaggio:** intestazione con dati principali e sei schede (Tappe, Attività, Checklist, Spese, Timeline, Valigia).
- **Form di inserimento/modifica:** viaggio, tappa, attività e spesa.
- **Dettaglio tappa:** informazioni della tappa ed elenco attività collegate.
- **Statistiche:** riepiloghi globali e per singolo viaggio, con grafici.

5.3 Flusso di navigazione

La navigazione principale avviene tramite una **NavigationBar** a due voci (Viaggi, Statistiche). Dalla lista dei viaggi si accede al dettaglio (passando il **tripId**); il dettaglio organizza le funzio-

nalità del viaggio in schede. Da ogni scheda si aprono i relativi form tramite `Navigator.push`, con ritorno alla vista precedente al termine dell'operazione.

```
Viaggi → Dettaglio viaggio → {Tappe, Attività, Checklist, Spese, Timeline, Valigia}
Tappe → Dettaglio tappa → Form attività
Statistiche (vista globale)
```

5.4 Organizzazione dei dati

Il modello relazionale prevede il viaggio come entità centrale a cui sono collegate, con relazioni uno-a-molti, tappe, attività, checklist e spese. Le foreign key sono configurate con `ON DELETE CASCADE` (eliminazione a cascata dei dati collegati al viaggio) e `ON DELETE SET NULL` per l'associazione facoltativa attività/checklist → tappa.

Relazione	Cardinalità	Note
Trip – Stage	1 : N	una tappa appartiene a un solo viaggio (cascade)
Trip – Activity	1 : N	attività del viaggio (cascade)
Stage – Activity	1 : N	associazione facoltativa (set null)
Trip – Checklist	1 : N	checklist del viaggio (cascade)
Checklist – Item	1 : N	elementi della checklist (cascade)
Trip – Expense	1 : N	spese del viaggio (cascade)

6 Scelte tecnologiche

6.1 Framework

L'app è sviluppata in **Flutter** (Dart), come richiesto dalla traccia. Flutter permette lo sviluppo cross-platform da un'unica base di codice e offre un ricco insieme di widget Material 3, adatti a realizzare un'interfaccia coerente e moderna.

6.2 Librerie principali

Libreria	Motivazione
provider	Gestione dello stato semplice e idiomatica basata su <code>ChangeNotifier</code> .
sqflite / sqflite_common_ffi_web	Database SQLite locale su mobile e supporto Web via <code>WebAssembly</code> .
uuid	Generazione di identificatori univoci per le entità.
intl	Formattazione di date e valuta secondo la localizzazione italiana.
fl_chart	Grafici (torta delle spese) nella dashboard statistiche.
collection	Utility sulle collezioni (es. <code>firstWhereOrNull</code>).

6.3 Motivazione delle scelte

È stato scelto `provider` per la sua semplicità e per l'ottima integrazione con il ciclo di vita dei widget, evitando la complessità di soluzioni più pesanti non necessarie per la dimensione del progetto. `sqflite` è lo standard di fatto per la persistenza relazionale in Flutter e soddisfa il requisito di funzionamento offline tramite un database SQLite locale, senza alcun servizio di rete.

6.4 Gestione della persistenza

Lo schema del database è creato alla prima apertura da `DatabaseHelper` (pattern singleton), con tabelle, vincoli di integrità referenziale e indici sulle foreign key per efficienza. L'accesso ai dati è mediato dalle repository, che espongono operazioni CRUD e query di ricerca/filtro. Al primo avvio, se il database è vuoto, viene caricato un insieme di dati demo.

7 Implementazione

7.1 Organizzazione del codice

Il codice è organizzato per livelli e per funzionalità (vedi struttura in precedenza). Ogni entità di dominio ha un modello con i metodi `toMap/fromMap` per la serializzazione verso/dal database e `copyWith` per la creazione di copie modificate in modo immutabile.

7.2 Gestione dello stato

Ogni area funzionale ha un proprio provider (`TripProvider`, `StageProvider`, `ActivityProvider`, `ChecklistProvider`, `ExpenseProvider`) registrato in un `MultiProvider` alla radice dell'app. I provider mantengono lo stato in memoria (per i dati collegati, in mappe indicizzate per viaggio) e notificano la UI con `notifyListeners()`. Le schermate osservano lo stato con `Consumer/context.watch` e invocano i metodi dei provider con `context.read`.

7.3 Gestione della navigazione

La navigazione usa il `Navigator` di Flutter con `MaterialPageRoute`. Il passaggio dei dati tra schermate avviene per parametri del costruttore (ad esempio `tripId`, `stageId`, o l'oggetto da modificare), garantendo un flusso esplicito e tipizzato. Il dettaglio del viaggio usa un `TabController` per le sei schede.

7.4 Gestione dei dati persistenti

Le repository incapsulano le query SQL. Ad esempio, l'eliminazione di un viaggio sfrutta l'integrità referenziale a cascata del database per rimuovere automaticamente tappe, attività, checklist e spese collegate, evitando cancellazioni manuali e stati incoerenti.

7.5 Parti significative

Duplicazione completa del viaggio

La difficoltà principale è mantenere coerenti i riferimenti tra le entità: duplicando le tappe si generano nuovi identificatori, quindi attività e checklist non possono conservare i vecchi `stageId`. La soluzione adottata fa restituire alla duplicazione delle tappe una mappa *id originale* → *id copia*, usata poi per ricollegare correttamente attività e checklist alle nuove tappe.

Listing 1: StageProvider: duplicazione delle tappe con mappa degli id

```
Future<Map<String, String>> duplicateStagesForTrip(
    String sourceTripId, String newTripId) async {
    final stages = await _repo.getByTrip(sourceTripId);
    final idMap = <String, String>{};
    for (final s in stages) {
        final newId = _uuid.v4();
        idMap[s.id] = newId; // mappa vecchio id -> nuovo id
        await _repo.insert(Stage(
            id: newId, tripId: newTripId, title: s.title, date: s.date,
            location: s.location, description: s.description,
            order: s.order, notes: s.notes,
        ));
    }
    await loadForTrip(newTripId);
    return idMap;
}
```

Listing 2: ActivityProvider: rimappatura della tappa associata

```
Future<void> duplicateActivitiesForTrip(
    String sourceTripId, String newTripId,
    Map<String, String> stageIdMap) async {
    final activities = await _repo.getByTrip(sourceTripId);
    for (final a in activities) {
        await _repo.insert(Activity(
            id: _uuid.v4(), tripId: newTripId,
            stageId: a.stageId == null ? null : stageIdMap[a.stageId],
            title: a.title, category: a.category, dateTime: a.dateTime,
            location: a.location, estimatedCost: a.estimatedCost,
            status: ActivityStatus.todo, notes: a.notes,
        ));
    }
    await loadForTrip(newTripId);
}
```

Packing list dal tipo di viaggio

La generazione della lista valigia è modellata da un'enumerazione `TripType` con un'estensione che espone gli oggetti specifici e un metodo che li combina con quelli di base. Alla richiesta di generazione, un menu inferiore fa scegliere il tipo e gli oggetti vengono aggiunti alla checklist evitando i duplicati.

Listing 3: TripType: costruzione della lista in base al tipo

```
extension TripTypeExtension on TripType {
    List<String> get specificItems { /* oggetti per tipo */ }
    List<String> buildPackingList() =>
        [...PackingTemplate.baseItems, ...specificItems];
}
```

Stato del viaggio calcolato

Lo stato mostrato all'utente non è un dato statico ma è calcolato dalle date rispetto al giorno corrente (futuro / in corso / completato), salvo lo stato "archiviato" impostato manualmente. Questo garantisce che l'elenco resti sempre coerente nel tempo senza aggiornamenti espliciti.

8 Conclusioni

L'applicazione realizza in modo coerente e integrato tutte le funzionalità richieste dalla traccia, con particolare attenzione all'usabilità (empty state, validazione, conferme) e alla coerenza dei dati (integrità referenziale, stato calcolato, duplicazione con rimappatura). Le due feature avanzate — duplicazione completa del viaggio e packing list generata dal tipo di viaggio — sono pienamente integrate nel flusso applicativo. Possibili sviluppi futuri includono l'introduzione di notifiche locali, l'esportazione/importazione dei dati per il backup e la suddivisione delle spese tra i partecipanti.